

An Empirical Study of Convergence and Reward Structures in Grid-World and Continuous Control MDPs

Saumya Rada

I. OVERVIEW OF MDPs

Blackjack and CartPole represent two fundamentally different classes of Markov Decision Processes (MDPs). Blackjack (Gymnasium’s Blackjack-v1) is a discrete, stochastic, and episodic MDP. The state space contains, the player’s current hand sum (ranging from 4 to 21), the dealer’s visible card (1 through 10), and a binary flag indicating if the player holds a usable ace. The state space contains 704 enumerable states. The action space contains two choices, stick (0) or hit (1). The reward structure is +1 for a win, 0 for a draw, and -1 for a loss, with no intermediate rewards during play. The environment is stochastic even under a deterministic policy as the outcome of any action depends on the unknown composition of the remaining deck. This means the transition function $p(s'|s, a)$ is a probability distribution over next states which makes the game stochastic. The stochasticity combined with terminal-only rewards creates high per-episode variance that slows convergence, thus requiring sustained exploration. A fast ϵ decay risks leaving infrequently visited states poorly evaluated. A conservative learning rate α is preferable since large updates amplify noise rather than accelerate learning. Because all returns are known at termination, γ should be kept near 1; lower values underestimate the long-run effect of actions taken early in a hand. This degrades policy quality in multi-step decisions.

CartPole (CartPole-v1) is deterministic given the same state and action. The physics simulation always produces the same next state but its state space is continuous and four-dimensional. The space is characterized by the cart’s position, velocity, the pole’s angle and the angular velocity. The action space is binary (push left or push right). The reward structure is +1 for every timestep the pole remains balanced, and the episode terminates when the pole tips beyond a threshold angle or the cart leaves the track boundary. The reward structure is dense and step-wise, in contrast to Blackjack’s terminal-only reward. The agent therefore receives a learning signal throughout the episode rather than only at the end. Because the state space is continuous, tabular methods require discretization, and bin resolution becomes a concern. Unlike Blackjack, where convergence is slowed by stochastic transitions and sparse rewards, CartPole’s deterministic dynamics and dense reward signal allow Q-values to update consistently each step, yet convergence remains sensitive to discretization resolution. Coarse bins group distinct states into the same cell, producing inconsistent updates and potentially unstable policies. However, fine bins may suffer from sparse visitation across the four-dimensional space.

Since CartPole is deterministic, Q-estimate variance arises not from environmental noise but from our discretization choices. Dense rewards remove the sparsity problem but can increase Q-values for stable states. A large α risks oscillation within the angular region required for balance. Because episodes can run up to 500 steps, a low γ will discount the value of long-run balancing, resulting in the policy being biased towards immediate balancing. Together, Blackjack and CartPole are interesting to study because their complexities are opposite. The first demands robustness to stochasticity and reward sparsity and the second demands robustness to representational choices in a deterministic but continuous world. This means that hyperparameter and design decisions that are optimal for one environment can be harmful in the other. This shows that RL algorithm design is problem-specific rather than universal.

II. HYPOTHESES

It is hypothesized that Policy Iteration (PI) will converge in substantially fewer outer iterations than Value Iteration (VI) on both environments, however this advantage will diminish for CartPole as the number of discretization bins increases. PI’s inner policy evaluation loop iterates until convergence under the current policy before each improvement step. This ensures each policy update is based on accurate value estimates. Therefore, the policy stabilizes in fewer outer rounds. VI updates values one sweep at a time and must perform many sweeps before the value function is close to the optimal policy. However, the efficiency gain per outer iteration is costly as the evaluation loop in PI becomes increasingly expensive as the state space grows.

Across all methods, it is hypothesized that Q-learning will achieve the highest final performance on CartPole, followed by SARSA, with VI and PI potentially underperforming when discretization is coarse or the model is inaccurate. This is because Q-learning’s off-policy updates allow it to directly approximate the optimal action-value function, even under exploratory behavior, leading to more aggressive and effective learning. SARSA, being on-policy, is expected to produce slightly lower but more stable returns due to incorporating the exploratory policy into its updates.

On Blackjack, it is expected that VI and PI will achieve the best overall policies due to access to a well-estimated transition model over a relatively small state space. Q-learning is hypothesized to achieve a marginally higher or comparable win rate than SARSA, since its off-policy updates allow it to learn the optimal policy more directly, while SARSA is expected to exhibit slightly lower variance and more

conservative behavior. Overall, model-based methods (VI/PI) are expected to perform best in smaller, well-modeled environments, while model-free methods (especially Q-learning) are expected to perform better in larger or more complex environments like CartPole where accurate models are harder to obtain.

III. METHODS

All details on hyperparameter tuning methods are found in the respective algorithm sections. The Blackjack experiment is conducted using the Gymnasium implementation of Blackjack-v1, formulated as a finite MDP. Value Iteration and Policy iteration are applied to the learned MDP model. VI iteratively updates the value function using Bellman optimality backups until convergence based on a predefined threshold, while PI alternates between policy evaluation and policy improvement. Both methods produce deterministic policies over the full state space, which are then evaluated by simulating episodes in the original environment. Performance is measured as the mean episodic return, averaged over multiple independent random seeds to account for stochasticity in card draws. Convergence behavior (e.g., number of iterations and value function change) is also tracked to compare computational efficiency between VI and PI.

For model-free approaches, tabular SARSA (on-policy) and Q-learning (off-policy) are trained over 50,000 episodes per seed using ϵ -greedy exploration with linearly decaying learning rates and exploration probabilities. SARSA updates action values using the next action actually taken under the current policy, while Q-learning updates toward the greedy optimal action regardless of behavior policy. Both methods maintain a Q-table over all state-action pairs and derive a policy by selecting the action with the highest estimated value. Training performance is monitored via episodic returns, and final policies are evaluated over 5,000 episodes per seed. All reported results are averaged across five seeds, with 95% confidence intervals computed to quantify variability and ensure robust comparisons across methods.

For both environments, on the model-based methods, convergence was determined by stability. Value Iteration (VI) terminated when the maximum change in the value function across all states, $\max_s |V_{k+1}(s) - V_k(s)|$, fell below a predefined threshold θ . Policy Iteration (PI) had a two rules, inner policy evaluation loop used the same value function stability check as VI, while the outer loop terminated when a full iteration produced a new policy identical to the previous one. For the model-free methods, SARSA and Q-learning, the convergence followed the running mean of the episode return. Training was considered to have converged and stopped when the running average did not improve by more than a small tolerance for 50 episodes. For all methods across both environments, their convergence pattern is reported through graphs.

On CartPole, all four methods operate on a discretized representation of CartPole-v1’s four-dimensional continuous state space (cart position, cart velocity, pole angle, and pole angular velocity). Each dimension is clipped to empirically

chosen bounds, (-2.4, 2.4), (-3.0, 3.0), (-0.25, 0.25), and (-3.5, 3.5) respectively, and divided into uniform bins using NumPy’s `digitize`. For the dynamic programming methods, a transition model is constructed empirically by rolling out 40,000–60,000 steps of epsilon-greedy exploration (ϵ decaying from 1.0 to 0.05 with a factor of 0.995 per step) and accumulating state-transition counts into per-action sparse matrices using `scipy.sparse`. Reward estimates are computed as running means over observed (s, a) pairs. Value Iteration and Policy Iteration then operate entirely on these sparse matrices, using `P.dot(V)` for efficient Bellman updates (they are applied using the same algorithm as Blackjack). For the model-free methods, SARSA and Double Q-Learning maintain separate `defaultdict` Q-tables updated online over 3,000 episodes, with epsilon decaying linearly from 1.0 to 0.01.

Hyperparameters for each method are identified via a three-stage successive halving search conducted before the discretization study, and are held fixed throughout it (details in hyperparameter tuning sections). The best hyperparameters are used in the discretization selection study which evaluates five candidate grid configurations, ranging from [1, 1, 6, 12] to [8, 8, 10, 12], across all four methods and all five seeds (42, 123, 256, 789, 1011), using 1,500 episodes for model-free methods and up to 60,000 transition samples for DP model construction. The configuration maximizing the combined mean return across all four methods is selected and used for all subsequent full training runs. The hardware class used for all experiments was CPU: Intel(R) Xeon(R) CPU @ 2.20GHz, OS Class: Ubuntu 22.04.5 LTS.

IV. VALUE ITERATION AND POLICY ITERATION HYPERPARAMETER TUNING

On Blackjack, Both Value Iteration and Policy Iteration used an identical three-stage successive halving protocol via a shared `staged_search` engine. In Stage 1, 24 configurations were sampled log-uniformly at random across two hyperparameters—discount factor $\gamma \in [0.90, 0.999]$ and initial value estimate $v_{\text{init}} \in [10^{-7}, 10^{-3}]$ —and each was evaluated over 200 episodes across 5 seeds (24,000 total episodes), after which the top 8 survivors were retained. Stage 2 tripled the per-configuration budget to 600 episodes $\times$ 5 seeds (another 24,000 episodes), pruning down to the top 3. Stage 3 performed local refinement around those 3 survivors by applying multiplicative perturbations to γ ($\times 0.5, 1.0, 2.0$) and v_{init} ($\times 0.75, 1.0, 1.25$), clipping to extended feasible bounds of $\gamma \in [0.80, 0.9999]$ and $v_{\text{init}} \in [10^{-9}, 10^{-2}]$. Total episode spend was approximately 55,000 per algorithm, with wall-clock times of 13.0s for VI and 14.4s for PI; the 1.4s gap reflects PI’s additional inner policy evaluation loop. Log-uniform sampling was chosen because both parameters span multiple orders of magnitude, where uniform sampling would severely under-explore the small- v_{init} and high- γ regimes that proved most important.

Both algorithms converged to identical champions ($\gamma = 0.95927$, $v_{\text{init}} = 3.8282$), as expected since VI and PI solve the same Bellman optimality equations on a fully known

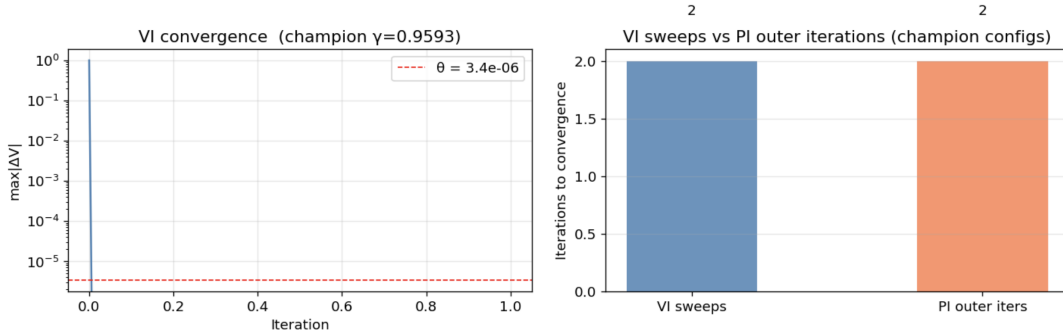


Fig. 1: Blackjack Value Iteration convergence and VI vs. PI iterations

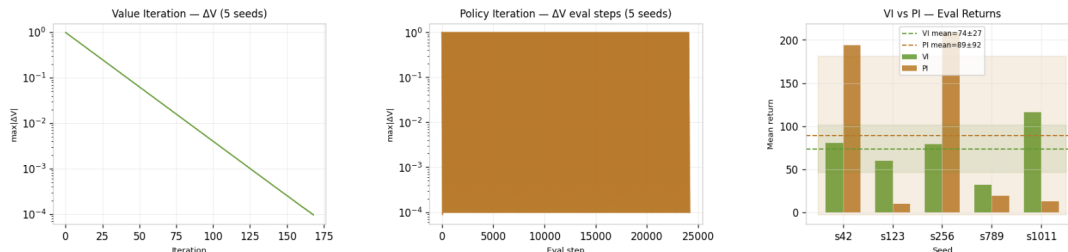


Fig. 2: CartPole Value Iteration convergence and VI vs. PI iterations

MDP. Of the two hyperparameters, γ showed more sensitivity. This is because values near the lower bound of 0.90 were consistently eliminated in Stage 1. The champion value near 0.96 reflects a meaningful trade-off between myopic and far-sighted behaviour. The initial value estimate v_{init} proved largely insensitive once set to a reasonable magnitude—the champion VI policy converged in only 2 iterations, indicating the MDP’s value function collapses to near-optimality almost immediately and that a broad range of v_{init} values yields the same policy. Stage 3 refinement produced no improvement over the Stage 2 best score of -0.0567 , confirming that the search had already located the optimum basin and that further tuning of either parameter in this neighbourhood returns diminishing gains.

Hyperparameter tuning followed the same 3-stage successive halving protocol as above for CartPole. Both VI and PI were tuned over two hyperparameters: the discount factor γ , sampled uniformly from $[0.90, 0.999]$, the value function initialization v_{init} , drawn as zero with probability 0.3 and otherwise sampled log-uniformly from $[10^{-2}, 10^1]$. The VI champion ($\gamma = 0.9463$, $v_{\text{init}} = 0.068$) scored 52.9 and the PI champion ($\gamma = 0.9259$, $v_{\text{init}} = 0.000$) scored 29.1, both means over 5 seeds $\times$ 100 episodes but at very different costs. VI completed the full search in 0.9 minutes while PI required 56.2 minutes. PI does not justify this overhead as it underperformed VI by 23.8 points. This makes VI the clearly preferable choice on both speed and quality.

Sensitivity analysis showed γ as the dominant parameter for both algorithms. A reduction in γ to 0.9179 cost VI 31.6 points and PI 9.3 points, while a small upward nudge to 0.9444 left PI nearly unchanged (+0.6 points), suggesting the PI champion sits near a local plateau in that direction.

For VI, increasing γ to 0.9653 similarly penalized performance by 20.6 points, indicating the champion occupies a relatively narrow γ optimum. The v_{init} parameter showed some sensitivity for VI. Halving v_{init} to 0.034 cost 4.1 points and doubling to 0.137 cost 11.6 points. However, v_{init} was entirely insensitive for PI, where both perturbations produced identical scores ($\Delta = 0.0$). This suggests v_{init} provides a small initialization bias for value iteration but has no effect on policy evaluation convergence once set to zero.

V. VALUE ITERATION AND POLICY ITERATION RESULTS

TABLE I: Blackjack Results

Method	Mean	$\pm 95\%$ CI	Wall(s)
VI	-0.0514	0.0135	14.3
PI	-0.0514	0.0135	15.2
SARSA	-0.0585	0.0106	67.9
Q-Learning	-0.0509	0.0089	68.6

On Blackjack, both Value Iteration (VI) and Policy Iteration (PI) achieve identical final performance, with a mean return of -0.0514 ± 0.0135 , confirming that both methods converge to the same optimal policy. The convergence behavior is extremely rapid as shown in Figure 1 where the Bellman residual drops from $\tilde{1.0}$ to below the θ threshold in a single iteration. The right panel of Figure 1 confirms this quantitatively — both VI and PI require exactly 2 iterations to convergence, with VI completing 2 full value sweeps and PI completing 2 outer policy improvement rounds. This fast convergence is expected given the structure of the Blackjack MDP, where rewards are largely immediate and episode lengths are short, allowing value information to propagate

across the state space almost instantly. The prediction that PI would converge in substantially fewer outer iterations than VI is not supported (both converge in 2 iterations). This is likely because of Blackjack’s compact state space and short horizons, a single Bellman sweep already propagates value information globally. This removes the advantage PI would otherwise gain from its inner policy evaluation loop. From a wall-clock perspective, the updated timings (14.3s for VI vs. 15.2s for PI) show that VI is marginally faster, this is due to PI’s overhead from the inner evaluation loop even when only 2 outer iterations are needed.

Overall, the results confirm that for compact, low-horizon MDPs like Blackjack, dynamic programming methods are highly robust. They converge almost immediately to the same optimal policy and are largely insensitive to hyperparameter choices across a wide range. Consequently, extensive hyperparameter tuning provides little practical benefit in this setting, and the theoretical distinctions between VI and PI, both in iteration count and computational efficiency, largely collapse in practice.

TABLE II: CartPole Results Summary

Method	Return (mean $\pm$ std)	Budget	Wall-clock
VI	73.7 $\pm$ 27.4	169 iters	2.34s
PI	88.9 $\pm$ 91.9	24,175 eval-steps	20.01s
SARSA	140.7 $\pm$ 9.5	3,000 episodes	61.1s
Q-Learning	180.5 $\pm$ 5.4	3,000 episodes	78.2s

On CartPole, both VI and PI perform worse than model-free methods as hypothesized. Value Iteration converged in 169 iterations, with $\max|\Delta V|$ decaying smoothly and linearly on a log scale from 10^0 to below 10^{-4} (as seen in Figure 2), consistent with the geometric contraction guaranteed by the Bellman operator. Policy Iteration, by contrast, never shows a visible ΔV decay across its 24,175 evaluation steps. In Figure 2, $\max|\Delta V|$ remains at 10^0 throughout, indicating that the policy evaluation phase fails to converge within the allocated budget. This is a known failure mode of PI when applied to discretized continuous-state spaces because the evaluation step requires solving an approximate linear system over an enormous and irregular state space, resulting in a poorly evaluated policy that cannot improve reliably. Despite this, PI achieves a higher mean return of 88.9 ± 91.9 versus VI’s 73.7 ± 27.4 , although PI’s much larger standard deviation shows that this mean is misleading. As shown in the per-seed bar chart in Figure 2, PI’s performance is dominated by two seed outliers at seed 42 and seed 256 (return ≈ 195 and ≈ 210) while reducing to near zero on seeds 123 and 1011. VI, by contrast, produces far more consistent results across seeds, with returns ranging from roughly 25 to 115, confirming it converges to a stable policy. The two algorithms do not converge to the same policy, VI yields reliable moderate performance while PI’s incomplete evaluation produces an erratic one. The wall-clock comparison further highlights the difference in performance. VI completes in 2.34s total, whereas PI requires 20.01s total, over $8\times$ slower, but both performed faster than the model-free methods (SARSA: 61.1s, Q-Learning: 78.2s),

which have substantially higher returns of 140.7 ± 9.5 and 180.5 ± 5.4 respectively at the cost of significantly longer training.

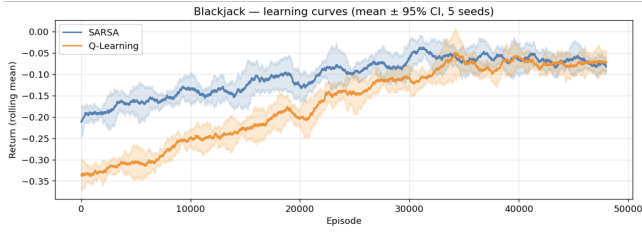
VI. SARSA AND Q-LEARNING HYPERPARAMETER TUNING

Both Q-Learning and SARSA were tuned over six hyperparameters jointly using the same three-stage successive halving protocol: initial exploration rate $\varepsilon_0 \in [10^{-2}, 1.0]$, exploration floor $\varepsilon_{\text{floor}} \in [10^{-3}, 10^{-1}]$, epsilon decay fraction $f_\varepsilon \in [0.1, 0.9]$, initial learning rate $\alpha_0 \in [10^{-3}, 1.0]$, final learning rate $\alpha_{\text{final}} \in [10^{-4}, 10^{-1}]$, and discount factor $\gamma \in [0.90, 0.999]$. The epsilon decay fraction specifies what fraction of the total training episodes are used to reduce ε from its starting value to its end value. All six were sampled log-uniformly, as uniform sampling would have over-represented large- α and large- ε configurations at the expense of the stable low-rate regimes that proved most effective. Stage 1 evaluated 24 candidates over 200 episodes $\times$ 5 seeds, pruning to the top 8; Stage 2 tripled the episode budget to 600×5 seeds, pruning to the top 3; Stage 3 applied multiplicative local refinement around the 3 survivors, perturbing α_0 and α_{final} by $\times\{0.5, 1.0, 2.0\}$ and f_ε by $\times\{0.75, 1.0, 1.25\}$. Total episode spend was approximately 55,000 per algorithm, with wall-clock times of 13.7s for SARSA and 14.2s for Q-Learning, comparable to VI and PI despite the added per-episode Q-table training cost, reflecting the modest MDP size where environment interaction time dominates over table update overhead.

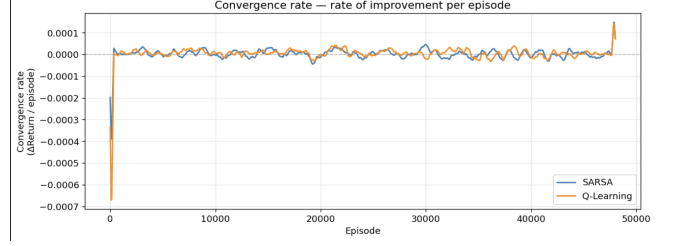
The champion configurations differed most sharply in α_0 and ε_0 : Q-Learning selected $\alpha_0 = 0.427$ and $\varepsilon_0 = 0.810$, which are much larger than SARSA’s $\alpha_0 = 0.053$ and $\varepsilon_0 = 0.360$, consistent with Q-Learning’s off-policy greedy backup benefiting from high early exploration while SARSA’s on-policy updates are already softened by the behavioural policy. Both algorithms converged on similar γ values near 0.96–0.98 and similarly small α_{final} and $\varepsilon_{\text{floor}}$ values, confirming conservative late-training behaviour in both cases. Q-Learning showed substantially larger gains in Stage 3 refinement than SARSA, suggesting its optimum is sharper and more sensitive to precise schedule tuning. Of the six parameters, α_0 and γ were the clearest performance drivers. Low values of either were consistently eliminated in Stage 1, while $\varepsilon_{\text{floor}}$ and α_{final} were effectively noise once set to sufficiently small values.

The same search protocol was used on CartPole. The hyperparameters tuned were: the learning rate α , sampled log-uniformly from $[10^{-3}, 1.0]$ to give equal representation across three orders of magnitude, and the epsilon decay fraction edf, sampled uniformly from $[0.2, 0.9]$, which controls what fraction of training episodes are spent decaying exploration. The sensitivity sweep perturbed α by $\times 0.5$ and $\times 2.0$, and edf by $\times 0.75$ and $\times 1.25$, around each champion using the same 5-seed, 800-episode budget.

The SARSA champion ($\alpha = 0.3040$, edf = 0.64) scored 129.1 and the Q-Learning champion ($\alpha = 0.0202$, edf = 0.64) scored 55.4, both means over 5 seeds $\times$ 800 episodes, with



(a) Blackjack Q-learning and SARSA returns



(b) Blackjack Q-learning and SARSA convergence

Fig. 3

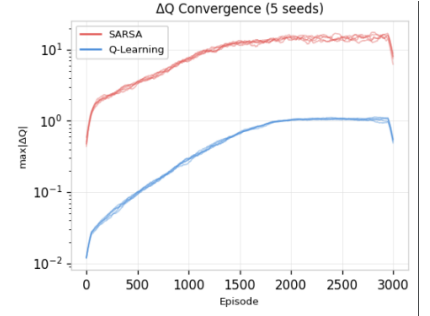
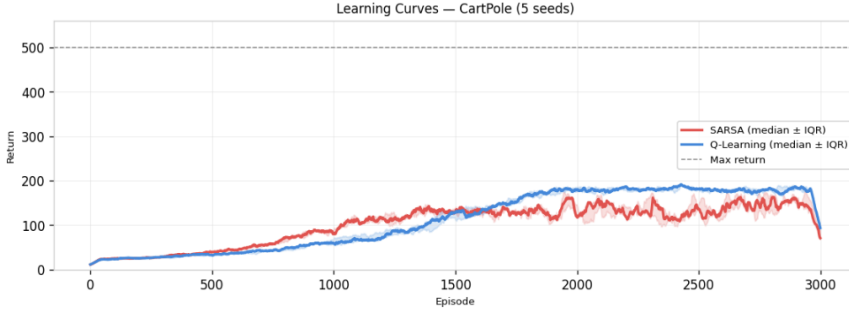


Fig. 4: CartPole SARSA and Q-learning returns/convergence

SARSA completing its search in 3.8 minutes and Q-Learning in 2.4 minutes, this shows similar per-episode costs between the two on-policy and off-policy updates. SARSA more than doubled Q-Learning’s final score, and the SARSA champion’s optimal α (0.3040) is roughly 12 times larger than Q-Learning’s (0.0202). This implies the performance gap is larger than the difference in completion time. Q-Learning’s off-policy updates seem to be destabilised by aggressive learning rates in this environment unlike SARSA’s on-policy updates. Sensitivity analysis showed both parameters matter for SARSA, as halving α cost 25.9 points while doubling it cost 22.0 points, and reducing edf below 0.64 to 0.48 increased points by 6.4 while increasing it to 0.80 improved the score by 3.3 points, suggesting the champion sits near a flat optimum for exploration. For Q-Learning, edf was important, since increasing it from 0.64 to 0.80 collapsed the score by 22.1 points, indicating that over-extended exploration hurts the off-policy updates, while reducing edf to 0.48 increased score by 15.5 points. Halving α to 0.0101 reduced the score by 5.3 points and doubling it to 0.0404 cost 17.6 points, confirming Q-Learning’s known sensitivity to step size and suggesting the champion sits slightly above optimal. Combined, the two model-free searches took 6.2 minutes. This is much faster than PI alone while SARSA’s champion score far exceeded both DP methods as well.

VII. SARSA AND Q-LEARNING RESULTS

On Blackjack, SARSA and Q-learning were each tuned via successive halving over their respective hyperparameter spaces. SARSA converged to a conservative exploration profile ($(\epsilon_0 = 0.360, \epsilon_{\text{floor}} = 0.025, \text{decay_frac} = 0.612, \alpha_0 = 0.053, \alpha_{\text{final}} = 0.002, \gamma = 0.981)$) while Q-learning

adopted a far more aggressive one ($\epsilon_0 = 0.811, \epsilon_{\text{floor}} = 0.016, \text{decay_frac} = 0.676, \alpha_0 = 0.428, \alpha_{\text{final}} = 0.010, \gamma = 0.961$). Both SARSA and Q-learning use ϵ -greedy exploration. Q-learning’s high initial ϵ and large α reflect its off-policy nature, where aggressive early exploration and fast value updates are less destabilizing because the target policy is greedy regardless of the behavior policy. SARSA, being on-policy, requires more careful exploration, large ϵ directly affects the policy being evaluated, so a lower initial exploration rate and slower learning rate help maintain stable policy improvement.

In terms of final performance, Q-learning achieves a mean return of -0.0509 ± 0.0089 , marginally outperforming SARSA at -0.0585 ± 0.0106 , which is consistent with the hypothesis that Q-learning’s off-policy updates allow it to learn the optimal policy more directly. However, the margin is small, and both methods fall short of the VI/PI benchmark of -0.0514 , confirming the broader hypothesis that model-based methods outperform model-free ones in small, well-modeled environments. As shown in Figure 3a, Q-learning converges to a slightly higher mean return than SARSA over 50,000 episodes, with both methods stabilising after approximately 20,000 episodes. The convergence rate plot in Figure 3b confirms that neither algorithm exhibits sustained improvement in the latter half of training, with per-episode gains oscillating near zero.

The hypothesis that SARSA would exhibit lower variance and more conservative behavior is only partially supported. SARSA’s wider confidence interval in Figure 3a indicates that it is less consistent across seeds than Q-learning. This suggests that the tuned hyperparameters matter more than the algorithm here. Both model-free methods required sub-

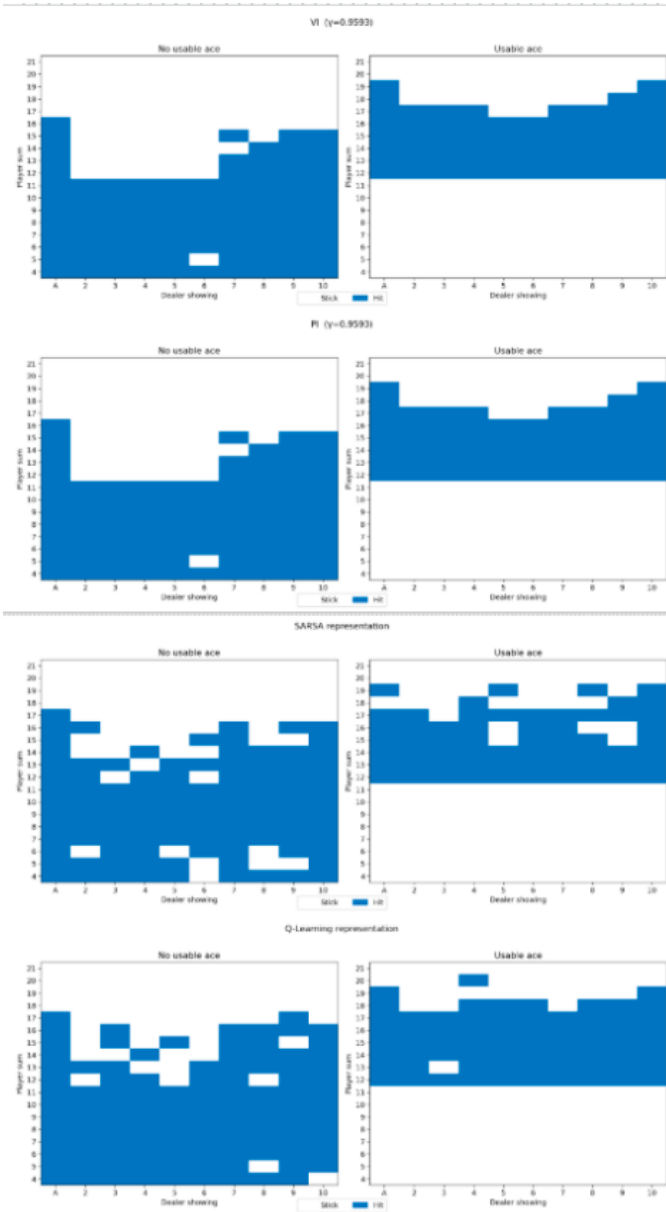


Fig. 5: Blackjack Heatmaps)

stantially more wall time (67.9s and 68.6s respectively) than VI and PI (14.3s and 15.2s). On a compact MDP like Blackjack where a near-perfect transition model is available, the additional computational cost of model-free exploration yields no performance benefit and in fact produces slightly worse results. This aligns with established reinforcement learning theory, where model-based methods exploit a known transition model to compute optimal policies directly, while model-free methods rely on sample-based updates that are typically less sample-efficient and require substantially more interaction to converge (Sutton & Barto, 2018). [1]

On CartPole, Q-Learning substantially outperforms SARSA in terms of mean return and both perform better than VI and PI as hypothesized, achieving 180.5 ± 5.4 against 140.7 ± 9.5 over 3,000 episodes, neither reaching

the maximum return of 500. Surprisingly Q-learning has a lower variance showing better stability unlike the hypothesis expected. Both methods use linear ϵ -greedy decay, and their tuned `eps_decay_frac` values are identical. SARSA retains exploration for 64% of training ($\alpha = 0.3040$) and Q-learning for 64% ($\alpha = 0.0202$), suggesting the performance gap is driven more by the learning rate and update rule than by exploration schedule differences.

In Figure 4 (left), both methods improve together through episode 1,000, after which Q-learning’s median diverges upward, reaching peaks near 180 before collapsing back, while SARSA plateaus after erratically varying around 100–120 with a wider IQR throughout. Q-learning has a slightly narrower shaded band and a small spike-and-collapse pattern, though its lower variance (± 5.4) compared to prior runs suggests more consistent behaviour. Overall, while Q-learning performs better, both methods do not reach the maximum of 500 at any point. SARSA’s near-flat IQR band demonstrates its stability.

The ΔQ convergence plot (Figure 4, right) shows SARSA’s updates grow *larger* than Q-learning’s, with the maximum ΔQ stabilising near 10–12 compared to Q-learning’s 1–2 by episode 3,000. This reflects the on-policy nature of SARSA’s updates, as the TD target is computed under the same ϵ -greedy policy being followed. Q-learning’s off-policy targets produce smaller but noisier per-episode updates. The update magnitude does not result in better performance. Q-learning’s smaller and more aggressive off-policy corrections have higher returns, and SARSA’s larger on-policy updates converge to a slightly worse but stable solution. Overall Q-learning dominates SARSA in performance and stability in this setting.

VIII. BLACKJACK’S POLICY HEATMAPS

The policy heatmaps in Figure 5 provide insight into how each algorithm learned behavior compares across the four methods. The VI and PI heatmaps are visually identical, which is expected given that both converge to the same optimal policy under the same champion configuration ($\gamma = 0.9593$). Both show a detailed and context-sensitive strategy, they hit aggressively at low player sums and shift to sticking at higher sums, with the boundary shifting depending on the dealer’s showing card. The usable-ace panels are more exploratory since both VI and PI continue hitting at sums of 17 and 18 in certain dealer configurations, reflecting the additional safety the ace provides. This behavior occurs only with access to a full transition model, as it requires accurate expected-value estimates in relatively rare states.

SARSA and Q-learning converge to broadly similar policies to each other, and their no-usable-ace panels resemble the VI/PI solution, hitting at low sums and sticking at high sums. However, both model-free policies are noisier, with scattered inconsistencies that reflect the stochasticity of sample-based learning over 50k episodes. These inconsistencies are more pronounced for SARSA than Q-learning, consistent with SARSA’s wider confidence interval and that

its conservative settings of α and ϵ limited convergence in regions of the state space not frequently visited.

The most significant divergence appears in the usable-ace panels. Both SARSA and Q-learning default to stick far earlier and more uniformly across dealer card configurations compared to VI and PI, producing a flatter, less differentiated policy in that region. This occurs because usable-ace states are visited less frequently during exploration, meaning Q-value estimates for those states are noisier and converge slower. Since VI and PI have access to the full transition model, they always compute exact expected values for these states, which is precisely the model-based advantage the hypothesis anticipated. The inability of model-free methods to recover the fine-grained usable-ace policy illustrate why VI and PI attain better mean returns despite only a small gap. This difference compared to SARSA and Q-learning is due to a infrequently visited region of the state space.

IX. DISCRETIZATION EFFECT ON CARTPOLE

The results show that coarser discretization consistently outperformed finer grids in the CartPole environment, making the smallest state space ($1 \times 1 \times 6 \times 12$, 364 states) the best configuration. All four algorithms achieved strong returns with these dimensions (Q-Learning: 155.1, SARSA: 137.7, VI: 91.9, PI: 93.3) as seen in Figure 6, with the $3 \times 3 \times 6 \times 12$ grid being the only configuration to slightly surpass it for SARSA. Beyond that point, performance reduced significantly. Policy Iteration collapsed to 28.4 at $6 \times 6 \times 8 \times 12$, Q-Learning dropped to 38.3, and Value Iteration reduced from 91.9 to 53.6 at the $4 \times 4 \times 8 \times 12$ grid before partially recovering. This is due to the nature of CartPole. Since the pole angle and angular velocity are well-represented by the 6×12 binning, extra binning is unnecessary. Increasing cart position and velocity bins further divides the state space without adding meaningful information. This causes sparse visits in model-free methods and slower value propagation in dynamic programming approaches.

The computational cost trade-off supports this conclusion as well. PI's runtime increased from 17.0 seconds to 499.4 seconds at the largest grid, and combined performance dropped nearly 32%. VI was the most efficient, it completed in under 0.5 seconds across all grid sizes, though even it reduced performance due to over-discretization. Taken together, the evidence strongly suggests that for CartPole, the relationship between state space size and solution quality is not one of diminishing returns but of reduced returns. Finer grids introduce noise and sparsity that destabilize learning. This makes compact, well-targeted discretization the computationally better strategy.

The computational cost of finer discretization has a trade-off between efficiency and accuracy. VI remained fast across all configurations, completing in under 0.4s regardless of grid size. VI achieved its best performance at the coarsest grid, with a mean return of 91.9 at $1 \times 1 \times 6 \times 12$, declining to 53.6 at $4 \times 4 \times 8 \times 12$ before partially recovering to 72.8 at the largest grid. PI scaled poorly in contrast, since its runtime grew from 17.0s at 364 states to 499.4s at 11 583 states, a

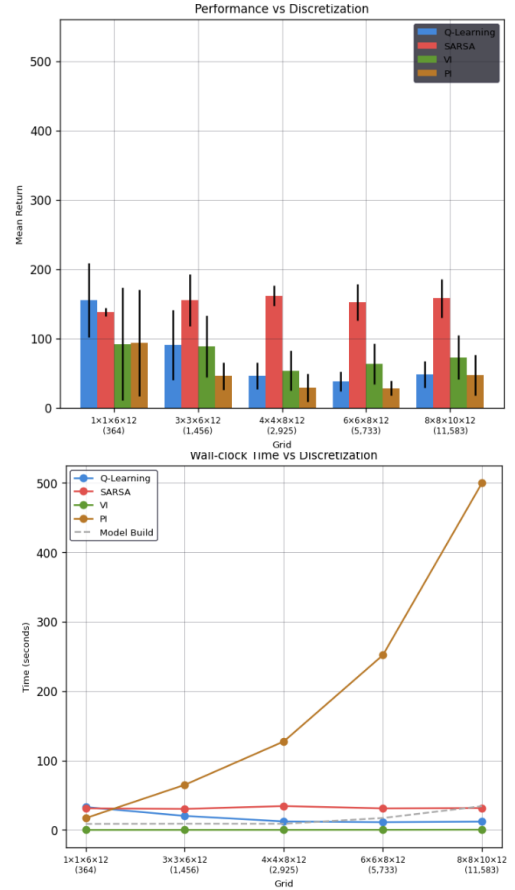


Fig. 6: Discretization effect across algorithms for CartPole

29 $\times$ increase, for a mean return of only 47.2, the lowest of the four algorithms at the largest grid. Model-free runtimes were largely flat (Q-Learning: 11–33 s; SARSA: 31–34 s). Model-free methods have a fixed episode budget rather than state-space dimensions, so finer grid configurations led to no extra computational cost for those methods.

X. DQN RESULTS

TABLE III: Performance comparison of Tabular Q-Learning and DQN variants across Blackjack-v1 and CartPole-v1. (5 seeds))

Env	Method	Mean Return	Uncertainty	Wall-clock
3*Blackjack	Tabular QL	-0.051	± 0.009 (95 % CI)	68.6 s
	Vanilla DQN	-0.008	± 0.036 (std)	50.1 s
	Double DQN	-0.022	± 0.027 (std)	44.0 s
3*CartPole	Tabular QL	180.5	± 5.4 (std)	123.7 s
	Vanilla DQN	157.2	± 170.8 (std)	14 m 7 s
	Double DQN	79.1	± 34.4 (std)	11 m 42 s

The DQN implementation uses a three layer fully connected Q-network shared across both environments with environment-specific preprocessing normalizing Blackjack's discrete tuple observation and passing CartPole's continuous state directly. A separate target network, a frozen copy of the online network synced every 200 steps, provides stable TD targets, preventing the moving-target problem inherent

in standard Q-learning. The Double DQN variant decouples action selection from evaluation, the online network selects the greedy next action while the target network evaluates it. This reduces the overestimation bias that Vanilla DQN accumulates through the max operator. Exploration follows a linear epsilon schedule from 1.0 down to a minimum over a fraction of total steps, and gradients are clipped at norm 10 to prevent instability.

Training stability and convergence presented distinct challenges across environments. On Blackjack, both variants converged to near-zero mean returns (Vanilla: -0.008, Double: -0.022), close to optimal for this high-variance card game, with tight standard deviations confirming consistent learned policies. Both outperformed tabular Q-learning (-0.051), as DQN’s function approximation generalizes across similar states whereas tabular methods must visit every state-action pair independently. CartPole was harder to stabilize, as Vanilla DQN averaged 157 but with a standard deviation of 170 (one seed hit 493 while others stalled below 110). This implies sensitivity to early exploration luck, while Double DQN was more consistent (mean 79, std 34) but converged lower, suggesting reduced overestimation bias slowed the optimistic exploration. Q-learning dominated all methods on CartPole, reaching 180.5 in just 3,000 episodes and 11 minutes total versus roughly 14 minutes per DQN variant. The discretized state representation of Q-learning reduces the problem to a tractable lookup rather than learning a continuous value function from scratch. The three-stage hyperparameter search completed in under 70 minutes, identifying epsilon decay and learning rate as the dominant factors. Vanilla DQN converged to a champion configuration of $\alpha = 1.36 \times 10^{-2}$, $\gamma = 0.9853$, $\varepsilon_{\text{end}} = 5.01 \times 10^{-3}$, and an epsilon decay horizon of 45.1% of training. Double DQN settled on a lower learning rate of $\alpha = 5.33 \times 10^{-3}$ with $\gamma = 0.9879$, $\varepsilon_{\text{end}} = 4.42 \times 10^{-2}$, and a shorter decay horizon of 30.4%, the higher terminal ε and compressed decay schedule reflect Double DQN’s reduced overestimation bias, which tolerates more residual exploration without destabilising the value targets. DQN is advantageous where the state space is too large or continuous for a table, but its additional complexity, replay buffers, target networks, and sensitive hyperparameters introduces variance that tabular methods avoid. As a result, deciding on the correct model is primarily dependent on state-space structure and tolerance for variance.

XI. CONCLUSION

Across both environments, the experiments confirmed that no single reinforcement learning rule dominates. The right method depends on the structure of the problem. On Blackjack, the hypothesis that model-based methods would outperform model-free ones in small, well-modeled environments was broadly confirmed, with VI and PI converging rapidly to the optimal policy while SARSA and Q-learning trailed. The model-free methods main disadvantage was in infrequently visiting regions of the state space. On CartPole, the hypothesized advantage of model-free methods in larger, harder-to-model environments was supported. Q-learning achieved

the highest overall returns and SARSA converged reliably, both substantially outperforming the dynamic programming methods. Q-learning exhibited higher variance than SARSA, suggesting that off-policy updates are harder to stabilize.

Several challenges emerged in the study. Policy iteration scaled poorly to CartPole’s larger state space, with its inner evaluation loop having very high computation for unreliable gains. This limitation is well documented in the literature, where Policy Iteration’s repeated policy evaluation steps become computationally expensive in high-dimensional or continuous state spaces due to the curse of dimensionality (Lee & Sutton, 2021) [2]. Discretization proved to be consequential, finer grids actively degraded performance by introducing sparsity, demonstrating that more expressive state representations do not automatically yield better policies. DQN introduced additional complexity through replay buffers, target networks, and sensitive hyperparameters that inflated variance and runtime, ultimately underperforming tabular methods on CartPole where a well-chosen discretization reduced the problem to a tractable lookup. Possible improvements include adaptive discretization targeting only policy-relevant dimensions, tighter evaluation budgets for PI, and more robust DQN training schedules to reduce seed-to-seed instability.

The main insight is that model-based methods are highly effective when a compact, accurate transition model is available, but are not as good once state spaces grow and model estimation becomes noisy. Model-free methods fill this gap, though the choice between on-policy and off-policy updates results in a stability-versus-optimality tradeoff that is environment dependent. Taken together, these results reinforce that algorithm selection in reinforcement learning cannot be decoupled from the state-space structure, model availability, and tolerance for variance. Moreover, simplicity in compact discretization or well-tuned tabular methods often outperforms more complex alternatives.

AI Use Statement

ChatGPT and Google Gemini were used to generate templates for training and testing parts of the models. Additionally, after completing the report I used Google Gemini to correct spelling and grammar errors.

REFERENCES

- [1] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press. Chapter 4 (“Dynamic Programming”) and Chapter 6 (“Temporal-Difference Learning”) section 6.6 (SARSA vs Q-learning) [Online].<https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [2] Lee, J., & Sutton, R. S. (2021). Policy iterations for reinforcement learning problems in continuous time and space: Fundamental theory and methods. *Automatica*, 126, 109421. [Online].<https://doi.org/10.1016/j.automatica.2020.109421>
- [3] MDP: Sutton & Barto (2018), Reinforcement Learning: An Introduction (2nd ed.), Example 5.3 “Blackjack,” online book. Gym Environment: Blackjack-v1.
- [4] MDP: Barto, Sutton & Anderson (1983), “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics* 13(5):834–846, doi:10.1109/TSMC.1983.6313077. Gym Environment: CartPole-v1.